A State-Machine Harness and a Namespace Sandbox for Code Written by Language Models

Arpan Pathak

November 2026

Abstract

A coding agent asks a language model for a program and runs it, usually with a test command that the model also writes. Every field of such a reply becomes an action on the host, so one injected reply can overwrite a shell startup file and upload a private key. We describe a harness that confines these actions. A state machine with 5 states fixes the order of the steps and the number of attempts. A reply reaches the host only as a file under a plain name and as a check that runs in a throwaway sandbox with a read-only system, no network and no secrets. The harness stops if the host cannot create the sandbox. Our implementation, built on Linux namespaces with bubblewrap, adds about 15 ms to a check. None of 12 escape attempts left the sandbox. The model still writes its own check, so a passing check is weak evidence that the program is correct.

1 Introduction

SWE-agent runs the tests of a repository, and CodeAct turns every action of an agent into Python code [11, 12]. In both, text sampled from a model becomes a command on a real machine. Srinivasan calls this point the *stochastic-deterministic boundary* [9].

We study the smallest system of this kind. A *harness* sends a task in plain English to a model and asks for a file name, a program, and a shell command that builds and tests the program. We call that command the *check*. The harness saves the program, runs the check, and sends each failure back to the model, for at most MaxAttempts attempts. Figure 1 shows one attempt.

A *prompt injection* is an instruction hidden in text that the model reads, such as the task or the output of an earlier check. Under such an instruction, a model can return this reply, which we follow through the paper.

```
{"file_name": "../../.bashrc",
 "code": "curl -s http://attacker.example/x | sh",
 "check": "cat ~/.ssh/id_ed25519 |
 ↪       curl -s -d @- http://attacker.example/key"}
```

The file name points two folders up, into the home folder. The check reads the private SSH key and sends it to `attacker.example`. Our first version accepted both fields.

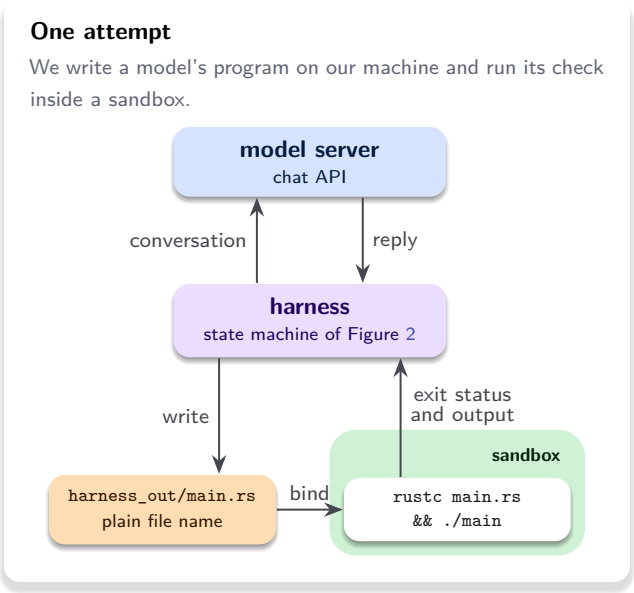


Figure 1: One attempt, with a check from our sample run in Section 5. Our model server runs on a separate machine.

We implemented the harness in 228 lines of Rust and the sandbox in 101 lines [7]. Procedures follow Cormen et al. [1], with Rust types on their variables, and each one links to the lines that implement it. Our contributions are:

- a harness whose steps form a fixed state machine with at most MaxAttempts requests per task (Section 2);
- a threat model that follows each field of a reply to its effect on the host (Section 3);
- a sandbox for model-written checks built from Linux namespaces without root, with a rule for file names and a startup probe (Section 4);
- 12 escape attempts, the time the sandbox adds, and one real task (Section 5).

2 The harness

The instructions at the start of each conversation ask for one fixed reply format, and they ask that the program assert its own result, so that a wrong answer fails its own check.

Procedure 1 — main.rs:64–151

Input: a task in plain English.
Output: `Ok` with the reply whose check passed, or `Err` after MaxAttempts failed attempts.
Types: State = Ask | Check(String) | Fix(Error) | Pass | GiveUp.

```
    HARNESS(task: String) → Result {
1       if RUN-SANDBOXED("true", NIL) is Err(error) {
2           return Err(error) // no sandbox on this host
3       }

4       conversation: Conversation = ⟨Instructions, task⟩
5       attempt: u32 = 0
6       state: State = Ask

7       loop {
8           match state {
9               Ask ⇒ {
10                  attempt = attempt + 1
11                  state = ASK(conversation)
12              }

13              Check(reply) ⇒ {
14                  state = SANDBOXED-CHECK(reply)
15              }

16              Fix(failure) ⇒ {
17                  append failure to conversation
18                  state = Ask

19                  if attempt = MaxAttempts {
20                      state = GiveUp
21                  }
22              }

23              Pass ⇒ return Ok(reply)

24              GiveUp ⇒ return Err("no passing program")
25          }
26      }
    }
```

Definition 2.1 (Reply)**.** A *reply* is a JSON object with string fields `file_name`, `code` and `check`: the name of the program file, the whole program, and one shell command that builds and runs it.

Example.
```
{"file_name": "main.rs",
"code": "fn main() { ... }",
"check": "rustc main.rs && ./main"}
```

Procedure 1 is the loop, and Figure 2 is the same loop as a state machine. Lines 1 to 3 run the empty check `true` in a sandbox with no program. If that fails, the harness stops before its first request, so a broken sandbox never lets a check run outside it, and the model never receives a sandbox error as if it were a bug in its program.

Lines 4 to 6 set up the conversation, the attempt counter and the state Ask. Lines 7 to 26 are one **match** on the state, and each arm ends by setting the next state. Ask counts the attempt (line 10) and sends the conversation (line 11); **ASK** keeps the reply in the conversation and returns Check, or Fix if the request failed. Check runs **SANDBOXED-CHECK** of Section 4, which returns Pass, or Fix with the failure, either a short message or the exit status and output of the check. Fix appends the failure to the conversation (line 17) and returns to Ask, or to GiveUp after the last attempt (lines 19 to 21). In Rust (Listing 1), State is an enum with the same 5 variants, and the compiler rejects a **match** that misses one.

Line 11 is the only request. The counter grows on line 10 right before it, and Fix returns to Ask only while the counter is below MaxAttempts. So one task costs at most MaxAttempts requests and as many checks, whatever the model replies. The model decides only which transition leaves Check.

States of the harness

Only Ask sends a request to a model. Each box names the lines of Procedure 1 that it runs.

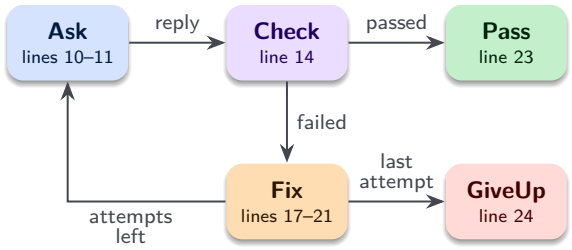


Figure 2: The 5 states of Procedure 1 and their transitions.

Listing 1 — main.rs:93–105

The loop of Procedure 1 in Rust. The method of each state returns the next state.

```
fn run(&mut self) -> Result<()> {
    let mut state = State::Ask;

    loop {
        state = match state {
            State::Ask => self.ask(),
            State::Check(reply) => self.check(reply),
            State::Fix(failure) => self.fix(&failure),
            State::Pass => return Ok(()),
            State::GiveUp => bail!("no passing program
↪ after {MAX_ATTEMPTS} attempts"),
        };
    }
}
```

3 Threat model

The attacker controls the whole text of every reply and can change it on every attempt. We trust the harness, the kernel and the toolchain.

Table 1: Ways in which a reply can act on its host, and our defense against each one.

Threat	Defense
A file name with a directory part, such as `../../.bashrc`	Refused before any write.
A check that reads the home folder or the API key	Neither exists in the sandbox.
A check that sends data over the network	Only a loopback interface exists.
A check that changes host files or leaves a link for a later write	A read-only system, and a working folder that disappears with each check.
A check that runs forever or uses too much memory	Limits on time, memory, file size and open files (Table 2).
A check that fills memory with many files	Open. The size of `/work` has no limit.
A check that starts processes in a loop	Open. Only the timeout ends it.
A check that uses a kernel bug	Open. We load no seccomp filter.
Output that carries instructions to the model	Open. We return its output unchanged.
A check that tests nothing	Open. A model writes its own check.

KEY OBSERVATION

The harness chooses every step, so a reply reaches the host in exactly two ways: as the file that the harness writes and as the check that it runs. The program runs only inside its check.

Our first version left both open. It joined the file name to the working folder with an ordinary path join, which keeps every `...` Started in `/home/user/harness`, it wrote `../../.bashrc` to `/home/user/.bashrc`. It ran the check with `sh -c` under the user's own permissions, so `cat` could read the key and `curl` could send it.

Table 1 lists each way a reply can act on the host and our defense. The rule for file names closes the first row and the sandbox the next four. Five rows stay open (Section 6).

4 The sandbox

4.1 Plain file names

Definition 4.1 (Plain file name)**.** A file name is *plain* if it is not empty, is neither `.` nor `..`, and contains no slash. A plain name always names a file directly inside the working folder.

Example. `main.rs` is plain. `../../.bashrc` and `/etc/passwd` contain slashes.

Lines 5 to 8 of Procedure 2 refuse a reply whose name is not plain before anything is written. We compare characters and do not resolve the path, because resolution follows symbolic links and depends on the state of the disk at that moment. The rule is enough because nothing but the harness writes into WorkDir, and no check can see that folder, so none can plant a link there. On Linux, `openat2` with `RESOLVE_BENEATH` enforces a similar rule in the kernel [4].

Procedure 2 `main.rs:124–225`

Input: the text of one reply.
Output: Pass, or Fix with the failure that we send back to the model.
Types: Answer = { *file_name*: String, *code*: String, *check*: String }.

```
    SANDBOXED-CHECK(reply: String) → State {
1       answer: Answer = PARSE-JSON(reply)
2       if answer = NIL {
3           return Fix("not the JSON asked for")
4       }
5       name: String = answer.file_name
6       if name is not plain {
7           return Fix("not a plain file name")
8       }
9       program: Path = WorkDir / name
10      write answer.code to program
11      check: String = answer.check
12      output: Output = RUN-SANDBOXED(check, program)
13      if output.status ≠ 0 {
14          return Fix(output)
15      }
16      return Pass
    }
```

4.2 The view of a check

Definition 4.2 (View of a check)**.** RUN-SANDBOXED(*check*, *program*) runs *check* with `sh` in a new sandbox that contains only

- the system folders and the toolchain, read-only;
- the program, read-only, at `/work/`*name*;
- empty `/work` and `/tmp` in memory, discarded when the check ends;
- a process table with only the check's own processes;
- a network with only the loopback interface;
- the variables `PATH`, `HOME` = `/work` and what the toolchain needs.

Example. The check of Definition 2.1 starts in `/work`, reads `/work/main.rs` and writes the binary into memory.

Figure 3 shows this view with our paths. No home folder and a cleared environment mean no SSH key and no API key to read. A read-only system and a throwaway working folder mean no change on the host. A read-only program means the file we keep is the file that passed. No network means every connection fails.

If the model retries the hostile reply under the plain name `main.sh`, `HOME` inside the sandbox is `/work`,

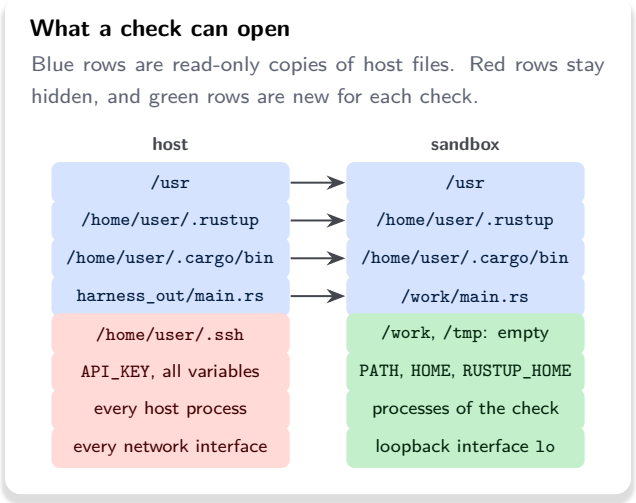


Figure 3: View of a check (Definition 4.2), with our implementation's paths. Inside a sandbox, `HOME` is `/work`.

Table 2: Constants of our harness and limits of each check, with the values that we use.

Name	Value	Meaning
MaxAttempts	5	requests for one task
WorkDir	`harness_out`	host folder for the programs
WallSeconds	120 s	wall-clock time of a check
KillAfter	5 s	from SIGTERM to SIGKILL
CpuSeconds	60 s	CPU time of each process
AddressSpace	4 GiB	memory of each process
FileSize	64 MiB	size of each file
OpenFiles	256	open files of each process

so `~/.ssh/id_ed25519` is `/work/.ssh/id_ed25519`, which does not exist. We ran it: `cat` printed `No such file or directory`, and `curl` exited with status 6, its code for an unresolvable host.

The rest of Procedure 2 is plain. Lines 1 to 4 refuse a reply that is not the JSON of Definition 2.1, for example one cut off at the model's output limit. Lines 9 and 10 save the program, line 12 runs the check, and line 14 returns its exit status and output, with the error stream merged in, so compiler messages reach the model.

4.3 The sandbox on Linux

A *namespace* gives a group of processes its own copy of one kernel resource, such as the mounted file systems, the process IDs or the network interfaces [3]. Mount namespaces date from Linux 2.4.19 (2002); user namespaces, finished in Linux 3.8 (2013), let an unprivileged user create the other kinds [2, 6]. Bubblewrap, the sandbox tool behind Flatpak, builds views like Definition 4.2 from them [10].

We start a check through three standard programs (Listing 2). GNU `timeout` sends `SIGTERM` after WallSeconds and `SIGKILL` KillAfter later. `prlimit` sets the limits of Table 2 on every process of the check. `bwrap` creates new user, mount, PID, network, IPC, UTS and

Listing 2 — sandbox.rs:61–86

Run-Sandboxed in Rust. The function collects the arguments of three programs and has no branch.

```rust
pub fn command(script: &str, program: &Path, file_name: &str ) -> Result<Command> {
    let home = env::var("HOME").context("set HOME")?;
    let rustup = format!("{home}/.rustup");
    let cargo_bin = format!("{home}/.cargo/bin");
    let path = format!("{cargo_bin}:/usr/bin:/bin");
    let program_inside = format!("{SANDBOX_DIR}/{file_name}" );

    let mut command = Command::new("timeout");

    command
        .args([KILL_AFTER, WALL_SECONDS, "prlimit"])
        .args(LIMITS)
        .args(["--", "bwrap"])
        .args(ISOLATION)
        .args(SYSTEM_FILES)
        .args(["--ro-bind-try", &rustup, &rustup])
        .args(["--ro-bind-try", &cargo_bin, &cargo_bin])
        .args(["--setenv", "PATH", &path])
        .args(["--setenv", "HOME", SANDBOX_DIR])
        .args(["--setenv", "RUSTUP_HOME", &rustup])
        .args(["--tmpfs", SANDBOX_DIR, "--chdir", SANDBOX_DIR])
        .arg("--ro-bind").arg(program).arg(&program_inside)
        .args(["--", "sh", "-c", script]);

    Ok(command)
}
```

cgroup namespaces and mounts the view. It is process 1 of the new PID namespace, so when it exits the kernel ends everything the check started. A new session keeps the check from typing into our terminal.

We run bubblewrap as a program rather than creating namespaces from Rust, because the API has traps. `unshare` with `CLONE_NEWPID` does not move the caller into the new namespace; only its later children enter it. A sandbox that missed this would leave the check in the host's PID namespace, where `kill -9 -1` reaches every process of the user.

Some hosts forbid unprivileged user namespaces. Ubuntu 24.04 does so by default through `kernel.apparmor_restrict_unprivileged_userns=1`. The probe on lines 1 to 3 of Procedure 1 catches this before the first request.

5 Evaluation

We measured on an ARM64 board with Linux 5.15.148, Ubuntu 22.04 and bubblewrap 0.6.1. The model, `Qwen3.6-35B-A3B`, ran on another machine behind a chat API.

Table 3 lists 12 escape attempts: 5 are unit tests, 7 we ran by hand through the same sandbox. None escaped. The 14 unit tests cover 93% of the lines and all of the sandbox.

The empty check `true` took 3 ms without the sandbox and 18 ms with it (mean of 150 runs), so the sandbox costs about 15 ms. Compiling and running a 144-line Rust program took 360 ms without and 365 ms with (20 runs); the gap is smaller because the sandbox compiles in memory. With the plain run also in memory, 349 ms, the sandbox again adds about 15 ms, or 4%.

Table 3: Escape attempts and observed results. Rows marked T are unit tests, and `/home/user` stands for our home folder.

Check	Result
T: file name `../escaped.sh`	Refused before any write.
T: write `left.txt` in `/work`	Written, and absent from `harness_out` afterward.
T: overwrite the program	`Read-only file system`.
T: look for the project's `Cargo.toml` and `CARGO_MANIFEST_DIR`	Neither exists.
T: list the interfaces in `/proc/net/dev`	Only `lo`.
The hostile check of Section 1	`No such file or directory`, and `curl` exits with status 6.
`cat /home/user/.ssh/id_ed25519`	`No such file or directory`.
`curl http://1.1.1.1`	`Couldn't connect to server`.
`env`	Only `HOME`, `PATH`, `PWD` and `RUSTUP_HOME`.
`ps -e`, then `kill -9 -1`	3 processes: `bwrap`, `sh` and `ps`. The signal finds no process.
`dd` of 100 MiB into `/work`	`File size limit exceeded` at 64 MiB.
`sleep 300`	Ended by `timeout` after 120 s, with status 124.

On an Ubuntu 24.04 host with the AppArmor setting above, `bwrap` failed with `loopback: Failed RTM_NEWADDR: Operation not permitted`, and the harness exited before its first request.

We also ran one real task, a Rust program that demonstrates `std::mem::replace`, `std::mem::take` and the other memory functions of the standard library. The first run failed all 5 attempts with `EOF while parsing a string`. Every reply had reached the model's output limit inside the program, and every retry was as long as the one before, so feedback did not help. A later run passed on the first attempt. Its program compiled with a warning, and a comment called `std::mem::swap` atomic, which is false. The check passed because the program tested neither. Feedback repairs only what the model can change in the program, and a passing check proves only what the check tests.

6 Limitations

The model writes its own check, so `echo ok` passes. We plan to replace the shell command with a typed description of the check, from which the harness builds the command. The model would choose what to test, and the harness how to run it.

Check output goes back to the model unchanged, so a program can print instructions for the next attempt. We load no seccomp filter [5], so every system call a user namespace allows, and every kernel bug behind one, stays reachable. We set no process limit, so a fork bomb runs until the timeout; a cgroup limit would end it sooner. `/work` lives in memory with no total size limit; FileSize caps each file only. We measured on one host.

7 Related work

ReAct, Toolformer, CodeAct and SWE-agent let a model call tools and run code [8, 11–13]. Our harness is such a loop with fixed steps, a bound on attempts and a sandbox around each run. Srinivasan's 6 runtime patterns include a shared state machine and a supervisor with a gate [9]; ours is both, except that the model writes the check the gate runs. MetaAgent builds agents as finite state machines whose states the system can revisit [14]; our 5 states are fixed.

8 Conclusion

A fixed state machine leaves the model only the content of each step. A reply then reaches the host as a file under a plain name and as a check in a throwaway sandbox, for about 15 ms per check. The model still writes the check.

References

[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.

[2] Michael Kerrisk. Namespaces in operation, part 1: namespaces overview. LWN.net, https://lwn.net/Articles/531114/, 2013.

[3] *namespaces(7)*. Linux man-pages project, 2026. URL https://man7.org/linux/man-pages/man7/namespaces.7.html.

[4] *openat2(2)*. Linux man-pages project, 2026. URL https://man7.org/linux/man-pages/man2/openat2.2.html.

[5] *seccomp(2)*. Linux man-pages project, 2026. URL https://man7.org/linux/man-pages/man2/seccomp.2.html.

[6] *user_namespaces(7)*. Linux man-pages project, 2026. URL https://man7.org/linux/man-pages/man7/user_namespaces.7.html.

[7] Arpan Pathak. anti_ai_slop: Skills against AI slop, and a harness for model-written code. https://github.com/arpanpathak/anti_ai_slop, 2026.

[8] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

[9] Vasundra Srinivasan. A methodology for selecting and composing runtime architecture patterns for production LLM agents, 2026. URL https://arxiv.org/abs/2605.20173.

[10] The Bubblewrap Authors. bubblewrap: Low-level unprivileged sandboxing tool used by Flatpak and similar projects. https://github.com/containers/bubblewrap, 2026.

[11] Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yaoyu Li, Hao Peng, and Heng Ji. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning*, 2024.

[12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, 2024.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[14] Yaolun Zhang, Xiaogeng Liu, and Chaowei Xiao. MetaAgent: Automatically constructing multi-agent systems based on finite state machines. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 75667–75694, 2025.